

FASE 1 · SEMANA 1



Imagem vs Container

Do zero absoluto até um container rodando no terminal — da forma mais profunda e simples possível

OBJETIVO DO DIA

Entender a diferença real entre Imagem e Container, e ter um container rodando no seu terminal.

[Por que Docker existe](#)[A analogia certa](#)[O que é uma Imagem](#)[O que é um Container](#)[Camadas](#)[Comandos essenciais](#)[Mão na massa](#)

CAPÍTULO 1 · O PROBLEMA

Por que Docker existe?

Antes de entender o que Docker é, você precisa sentir o problema que ele resolve. Sem isso, ele parece uma ferramenta desnecessariamente complexa.

A frase que matou projetos por décadas

"Na minha máquina funciona."

Você termina uma feature, funciona perfeitamente local. Sobe para o servidor do cliente e quebra. Por quê? Porque o servidor tem uma versão diferente do Node. Ou do .NET. Ou uma variável de ambiente com nome diferente. Ou um sistema operacional diferente. Ou um banco com configuração diferente.

O ambiente onde o código roda importa tanto quanto o código em si. E ambientes são frágeis, inconsistentes, e impossíveis de "versionar" do jeito tradicional.

O que Docker resolve

Docker cria um **pacote completo** que contém não só o seu código, mas o ambiente inteiro que ele precisa para rodar: o sistema operacional base, as dependências, as configurações, as variáveis de ambiente. Tudo junto, versionado, portátil.

A promessa do Docker: se roda na sua máquina, roda em qualquer lugar. Sem exceções. O ambiente viaja junto com o código.

Pense assim: em vez de entregar só o código do seu projeto e torcer para o servidor ter a versão certa do .NET, você entrega o projeto *mais* o .NET *mais* as

configurações *mais* o sistema operacional necessário, tudo empacotado em um único arquivo.

CAPÍTULO 2 · A ANALOGIA

A analogia certa para um desenvolvedor

Existem várias analogias para Docker. A mais comum é a da receita vs o prato. Funciona, mas é superficial. Vou te dar a analogia que faz mais sentido para quem programa.

Imagem é uma Classe. Container é uma Instância.



Imagem

É a **definição**. Um arquivo estático que descreve como o ambiente deve ser. Não roda. Não consome memória. Só existe como blueprint.



Container

É a **execução**. Uma imagem que foi "instanciada" e está rodando como processo vivo, com memória, CPU, e estado próprio.

Exatamente como em OOP:

```
analogia.ts
```

```
// IMAGEM = a definição da classe (não existe em memória  
ainda)
```

```
class MinhaApi {  
  runtime = "dotnet 8"  
  porta = 8080  
  env = { DB_HOST: "postgres" }  
}  
  
// CONTAINER = instâncias rodando em memória  
const container1 = new MinhaApi() // porta 8080  
const container2 = new MinhaApi() // porta 8081  
const container3 = new MinhaApi() // porta 8082  
  
// A CLASSE não muda. As instâncias têm estado e vida  
próprios.
```

Você pode criar quantos containers quiser a partir da mesma imagem. Cada um terá seu próprio estado, sua própria memória, seu próprio ciclo de vida — mas todos partem da mesma definição.

Regra de ouro: a imagem é imutável. Você nunca "edita" uma imagem em execução. Você edita o Dockerfile, gera uma nova imagem, e cria novos containers a partir dela.

CAPÍTULO 3 · IMAGEM

O que é uma Imagem — em profundidade

Uma imagem Docker é um arquivo binário composto por **camadas sobrepostas** (layers). Cada camada representa uma instrução do Dockerfile. Entender isso é fundamental para usar Docker de forma eficiente.

O Dockerfile — a receita que gera a imagem

Uma imagem não nasce do nada. Você a descreve em um arquivo chamado `Dockerfile`. Cada linha desse arquivo se torna uma camada da imagem.

Dockerfile

```
# Camada 1 – qual sistema operacional base usar
FROM mcr.microsoft.com/dotnet/aspnet:8.0

# Camada 2 – definir o diretório de trabalho dentro do
container
WORKDIR /app

# Camada 3 – copiar os arquivos do seu projeto para dentro da
imagem
COPY . .

# Camada 4 – expor a porta que a aplicação usa
EXPOSE 8080

# Camada 5 – o comando que roda quando o container iniciar
ENTRYPOINT ["dotnet", "MinhaApi.dll"]
```

Camadas — o que torna Docker inteligente

Cada instrução do Dockerfile cria uma camada. O Docker **cacheia** cada camada separadamente. Se você mudar apenas o seu código (Camada 3), ele não precisa baixar o .NET de novo (Camada 1) nem recriar o diretório (Camada 2) — apenas reexecuta a partir da camada modificada.

5 **ENTRYPOINT** — comando de inicialização

4 **EXPOSE** — porta da aplicação

3 **COPY** — seu código-fonte

2 **WORKDIR** — diretório de trabalho

1 **FROM** — sistema operacional base

Todas essas camadas são **somente leitura**. A imagem em si nunca muda. Quando você cria um container a partir dela, o Docker adiciona uma única camada extra

no topo — a camada de escrita.

Onde ficam as imagens?

Imagens ficam armazenadas em **registries**. O registry padrão é o **Docker Hub** — pense nele como o npm, mas para imagens Docker. Quando você executa `docker pull nginx`, está baixando a imagem oficial do nginx do Docker Hub para sua máquina.

Docker Hub

Registry público. Imagens oficiais do nginx, postgres, node, .NET, etc. Qualquer um pode publicar e baixar.

Registry privado

Sua empresa pode ter um registry próprio (AWS ECR, Azure ACR, GitHub Container Registry) para imagens internas.

Cache local

Imagens baixadas ou buildadas ficam em cache na sua máquina. `docker images` lista todas.

Tags

Versões de uma imagem. `nginx:latest`, `node:20`, `postgres:16-alpine`. Sempre especifique a versão em produção.

CAPÍTULO 4 · CONTAINER

O que é um Container — em profundidade

Um container é uma imagem em execução. Mas entender o que acontece por baixo muda como você trabalha com eles.

O container não é uma máquina virtual

Esse é o maior equívoco sobre Docker. Muita gente pensa que container = VM leve. Não é.

Característica	Máquina Virtual (VM)	Container Docker
Sistema operacional	Próprio, completo (kernel + tudo)	Compartilha o kernel do host
Tempo de inicialização	Minutos	Milissegundos a segundos
Tamanho típico	GBs	MBs
Isolamento	Completo (hardware virtualizado)	Processo isolado (namespace + cgroups)
Overhead de performance	Significativo	Quase zero

Um container é, na essência, um **processo isolado** rodando no seu sistema operacional. O Linux usa dois recursos para criar esse isolamento:

Namespaces

Fazem o container enxergar apenas seus próprios processos, rede, sistema de arquivos — como se fosse um OS próprio, mas não é.

cgroups

Limitam os recursos que o container pode usar: CPU, memória, I/O. Garante que um container não consuma tudo da máquina.

A camada de escrita — por que containers são efêmeros

Lembra que todas as camadas da imagem são somente leitura? Quando um container é criado, o Docker adiciona uma camada de escrita **temporária** no topo.

Camada de escrita (container) — temporária, destruída ao parar

R Camada 5 — somente leitura (da imagem)

R Camada 4 — somente leitura (da imagem)

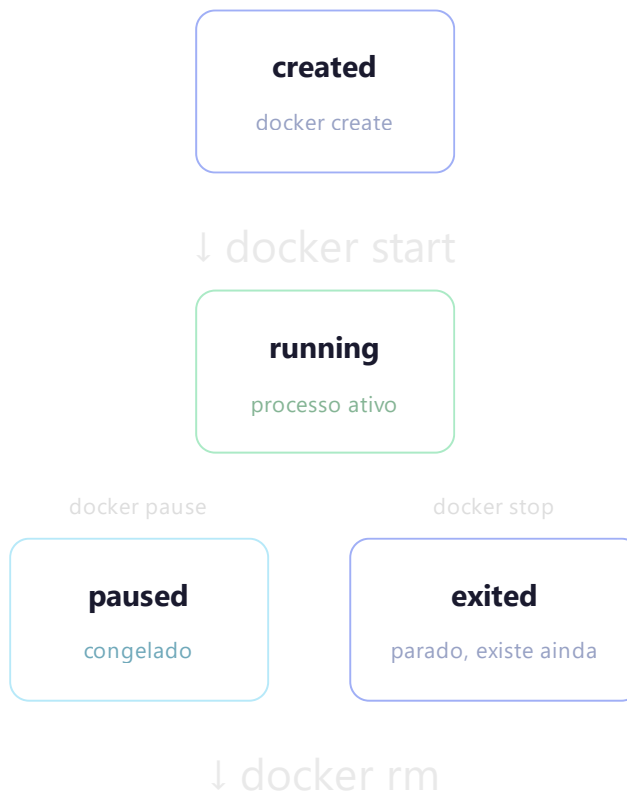
R Camada 3 — somente leitura (da imagem)

Implicação importante: qualquer dado que você escrever *dentro* de um container (logs, arquivos, dados de banco) some quando o container para. Para persistir dados, você usa **volumes** — aprenderemos isso na próxima semana.

Estados de um container

Um container tem um ciclo de vida. Entender os estados é essencial para debugar problemas.

CICLO DE VIDA DE UM CONTAINER



removed

destruído

Detalhe crucial: `docker stop` para o container mas não destrói. Ele continua existindo com seu estado. `docker rm` destrói de vez. Use `docker run --rm` para destruir automaticamente ao parar.

CAPÍTULO 5 · VISÃO COMPLETA

Como tudo se conecta

DO DOCKERFILE AO CONTAINER EM PRODUÇÃO

VOCÊ ESCRIVE

Dockerfile

instruções de build

↓ `docker build`

GERADO LOCALMENTE

Imagem

minha-api:v1.0

↓ `docker push` (opcional)

ARMAZENADO NO REGISTRY

Docker Hub / ECR / ACR

enzoagw/minha-api:v1.0

↓ docker run (em qualquer máquina)

RODANDO EM MEMÓRIA

Container 1

porta 8080

Container 2

porta 8081

Container 3

porta 8082

Mesma imagem, três containers. Cada um é um processo independente. Isso é a base do que escala em produção — orquestradores como Kubernetes fazem exatamente isso, só que com centenas de containers.

CAPÍTULO 6 · COMANDOS

Os comandos que você vai usar todo dia

Imagens

terminal

*# Baixar uma imagem do Docker Hub**\$ docker pull nginx**# Listar todas as imagens locais**\$ docker images*

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
------------	-----	----------	---------	------

nginx	latest	a72860cb95fd	2 weeks ago	187MB
-------	--------	--------------	-------------	-------

Buildar uma imagem a partir do Dockerfile da pasta atual

```
$ docker build -t minha-api:v1 .  
# ↑ nome:tag ↑ contexto (pasta atual)  
# Remover uma imagem  
$ docker rmi minha-api:v1
```

Containers

terminal

```
# Criar e iniciar um container  
$ docker run nginx  
# Flags mais importantes do docker run:  
$ docker run \  
-d \ # detached: roda em background (não trava o terminal)  
-p 8080:80 \ # porta: mapeia host:container  
--name meu-nginx \ # nome amigável  
--rm \ # remove automaticamente ao parar  
nginx # nome da imagem  
# Listar containers em execução  
$ docker ps  
# Listar TODOS (incluindo parados)  
$ docker ps -a  
# Ver Logs de um container  
$ docker logs meu-nginx  
$ docker logs -f meu-nginx # -f = follow (tempo real)  
# Entrar dentro de um container em execução  
$ docker exec -it meu-nginx bash  
# ↑ -i = interativo, -t = terminal  
# Parar um container  
$ docker stop meu-nginx  
# Remover um container parado  
$ docker rm meu-nginx  
# Parar E remover em um comando  
$ docker rm -f meu-nginx
```

O -p explicado

A flag `-p 8080:80` é o que mais confunde no início. Ela mapeia portas:

-p [porta-da-sua-máquina]:[porta-dentro-do-container]

O container vive em uma rede isolada. Ele não sabe que existe um "fora". Você precisa criar uma ponte: *"quando alguém bater na porta 8080 da minha máquina, encaminhe para a porta 80 dentro do container"*.

CAPÍTULO 7 · OBJETIVO DO DIA

Mão na massa — container rodando no terminal

Agora vamos executar. Siga cada passo na ordem. O objetivo é ter um container rodando e você enxergando o resultado no navegador.

Pré-requisito

Docker Desktop instalado e rodando. Se ainda não instalou: docker.com/products/docker-desktop

1 Verifique se o Docker está rodando

Abra o terminal e execute:

PowerShell

```
$ docker --version
```

```
Docker version 26.1.1, build 4cf5afa
```

```
$ docker ps
```

```
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
```

Se aparecer essa linha de cabeçalho, está funcionando.

2 Rode seu primeiro container — nginx

Vamos subir um servidor web nginx. O Docker vai baixar a imagem automaticamente se não tiver localmente.

PowerShell

```
$ docker run -d -p 8080:80 --name meu-nginx nginx
# O que cada parte significa:
# -d → roda em background (não trava o terminal)
# -p 8080:80 → mapeia porta 8080 da sua máquina → porta 80 do
container
# --name → dá um nome amigável ao container
# nginx → nome da imagem a usar
Unable to find image 'nginx:latest' locally
latest: Pulling from library/nginx
...
a72860cb95fd2b0f4a... ← ID do container criado
```

3 Confirme que está rodando

PowerShell

```
$ docker ps
CONTAINER ID IMAGE STATUS PORTS NAMES
a72860cb95fd nginx Up 3 seconds 0.0.0.0:8080->80/tcp meu-nginx
```

4 Abra no navegador

Acesse <http://localhost:8080> — você verá a página padrão do nginx. Um servidor web completo rodando em um container isolado na sua máquina.

5 Explore — entre dentro do container

PowerShell

```
# Entra no terminal do container (como se fosse SSH)
$ docker exec -it meu-nginx bash
root@a72860cb95fd:/# ← você está DENTRO do container agora

# Veja os processos que existem DENTRO desse container
root@a72860:/# ps aux
USER PID COMMAND
root 1 nginx: master process
nginx 29 nginx: worker process

# Só esses processos existem aqui. Isolamento total.

# Sair do container
root@a72860:/# exit
```

6 Veja os logs em tempo real

PowerShell

```
# Acompanhe os logs enquanto acessa localhost:8080 no
navegador
$ docker logs -f meu-nginx
172.17.0.1 - - [14/May/2026] "GET / HTTP/1.1" 200 615
172.17.0.1 - - [14/May/2026] "GET /favicon.ico" 404 153
```

7 Pare e limpe

PowerShell

```
$ docker stop meu-nginx
meu-nginx
$ docker rm meu-nginx
meu-nginx
$ docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
# Vazio — Limpo.
```



Objetivo do dia conquistado

Você rodou um container, explorou seu interior, acompanhou logs em tempo real, e o removeu corretamente. Você entende agora a diferença real entre imagem e container.

RESUMO

O que você aprendeu hoje

- ✓ Docker resolve o problema de "na minha máquina funciona" empacotando código + ambiente juntos
- ✓ Imagem = definição estática (como uma classe). Imutável, versionável, portátil
- ✓ Container = execução da imagem (como uma instância). Tem estado próprio, ciclo de vida próprio
- ✓ Imagens são compostas por camadas cacheadas — builds são rápidos porque só reexecutam o que mudou
- ✓ Containers compartilham o kernel do host — não são VMs, são processos isolados
- ✓ A camada de escrita do container é temporária — dados somem ao parar (volumes resolvem isso)
- ✓ Comandos essenciais: `docker run`, `docker ps`, `docker stop`, `docker rm`, `docker logs`, `docker exec`
- ✓ A flag `-p` mapeia porta da sua máquina para porta dentro do container

O que vem na próxima sessão

Dockerfile para o nexus-core

Escrever o Dockerfile real do seu projeto .NET e gerar a primeira imagem própria

docker-compose.yml

Subir API + banco de dados juntos com um único comando

Volumes

Persistir dados do banco fora do container — essencial para produção

Variáveis de ambiente

Configurar secrets e connection strings sem hardcode